

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

Revision History

Date	Version	Description	Author
15/SEP/26	1.0	This is the initial first draft to set up the Software Architecture documentation, primarily focusing on intent of document.	Kristoffer Roy Comahig
15/SEP/26	1.1	Expanded on Sections 2, 3, 4, and 6	Kristoffer Roy Comahig
17/SEP/26	1.2	Completed All Sections	Kristoffer Roy Comahig
18/SEP/26	2.0	Final Grammatical Edits	Kristoffer Roy Comahig
11/OCT/26	3.0	Project 2 maintenance: added game mode selection (Solo, Multiplayer, AI)	Joseph Ubanda, Tram Dinh

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

Table of Contents

1.	Introduction	4
1.1	Purpose	4
1.2	Definitions, Acronyms, and Abbreviations	4
1.3	References	5
1.4	Overview	5
2.	Architectural Representation	5
3.	Architectural Goals and Constraints	6
4.	Logical View	6
4.1	Overview	6
4.2	Architecturally Significant Design Packages	7
5.	Interface Description	13
6.	Quality	13

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

Software Architecture Document

1. Introduction

1.1 Purpose

This document outlines the architecture of Minesweeper, a puzzle coded in Python where players interact with a 10x10 grid, uncovering cells to reveal numbers indicating adjacent mines while avoiding detonation. Players can flag suspected mine locations. The player will be able to interact with this puzzle via a Command Line Interface to designate the cells of the grid they would like to interact with, whether that be by uncovering that spot or by flagging it. The game is won if all non-mine cells are uncovered without uncovering a mine and lost if a mine is uncovered at any point.

This version extends the inherited Project 1 codebase with these Project 2 features:

- Game modes: The player chooses Solo, Multiplayer, or AI. In Multiplayer, two players take turns on the same board. In AI mode, the player and an AI solver alternate turns on the same board. In Auto the AI solves the whole board without the players turns
- AI solver: The player selects a difficulty. Easy uncovers random covered, unflagged cells. Medium applies two rules based on revealed numbers: flag all hidden neighbors when a cell's number equals its hidden-neighbor count, and uncover the remaining hidden neighbors when its number equals its flagged-neighbor count. Hard adds the 1-2-1 pattern rule. Medium and Hard fall back to a random move when no rule applies.

1.2 Definitions, Acronyms, and Abbreviations

- Grid: A 10 x 10 array of cells that indicate places where a mine, flag, or nothing at all can be.
- Cell: an element within a 10x10 grid, represented as a List within Python. A cell's state or stored value is represented via a number range of 0-4 inclusive. The numbers represent the following states:
 - o 0 = Unchecked and no mine
 - o 1 = False Flag (flag on a non-mine cell)
 - o 2 = Checked and no mine
 - o 3 = Unchecked mine
 - o 4 = Flagged Mine
- Checked: a cell that has been uncovered and its state revealed. This is done by the user or AI interacting with said cell via left click. When a cell that has no adjacent mines is checked, adjacent cells that do not have mines are recursively checked or uncovered as well. The first cell the player or AI selects is guaranteed to be mine-free.
- Mine: A state a cell can be in, as randomly pre-determined during the board population that takes place when the user interacts with the game for the first time. When a cell of this state is interacted with, the following changes can occur: if the interaction was to flag it, it is flagged and the cell state is changed to 4. If the interaction was to uncover it, the game is lost.
- Flag: A state a cell can be in as well as an additional input mode within the user interface. Players can place/remove flags on covered cells to indicate potential mines. Flagged cells cannot be uncovered until unflagged.

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

- Game mode: Solo (one player), Multiplayer (two players alternate turns), or AI (the player and the AI solver alternate turns).
- Current player: The player whose turn it is, stored in `currentPlayer` (1 or 2).
- AI solver: The function `ai_solver(board, difficulty)` in `ai_solver.py`, which chooses and applies one AI move.
- Difficulty (Easy / Medium / Hard): The AI solver's strategy level. Easy is random, Medium applies the flag and uncover rules, and Hard adds the 1-2-1 rule.
- 1-2-1 pattern: Three side-by-side checked cells showing 1,2,1 the two outer hidden neighbors are mines, and the inner hidden neighbor is safe.

1.3 References

No other documents are referenced within this document beyond the official Python 3.14.7 documentation which can be found at the link below.

<https://docs.python.org/3/>

1.4 Overview

In this Software Architecture Document, the functionality of key modules (Board Manager, User Interface, Input Handler, Game Logic Handler, and AI Solver) are explained in-depth alongside the data flow as the player or AI works through the puzzle. The document includes the primary goals of those key modules, the core components and how they connect, along with the considerations for design decisions made.

2. Architectural Representation

The software architecture of Minesweeper is layered with each module serving to provide more functionality onto the Board Manager or Board.py. These modules are separated by what ownership they have over certain user stories or interactions are necessitated by the game such as user input, tracking the rules of the puzzle and applying them to the board, etc. Each module holds specific classes and methods to allow users to complete those actions and ensure the rules of the game are maintained. The order in which these modules are invoked in order to maintain the core gameplay loop is enforced by the Game Logic Handler.

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

3. Architectural Goals and Constraints

Goals:

- **Ease-Of-Use:** The architecture is designed to make usability as easy as possible, allowing the user to focus on the entertainment of the game itself rather than dealing with the intricacies and minutiae of the programming.
- **Efficiency & Speed:** The architecture is designed to make game state and board state updating as fast as possible, ensuring the tempo of the game isn't bogged down by the program itself.

Constraints:

- **Programming Language and Environment:** Minesweeper is developed in Python and across multiple environments that can support programming in Python such as Visual Studio Code.
- **Portability & Compatibility:** Minesweeper is designed to be compiled and run across different operating systems capable of running Python programming.
- **Schedule Constraints:** Minesweeper along with its corresponding documentation is slated to be completed by 9/20/2026.

4. Logical View

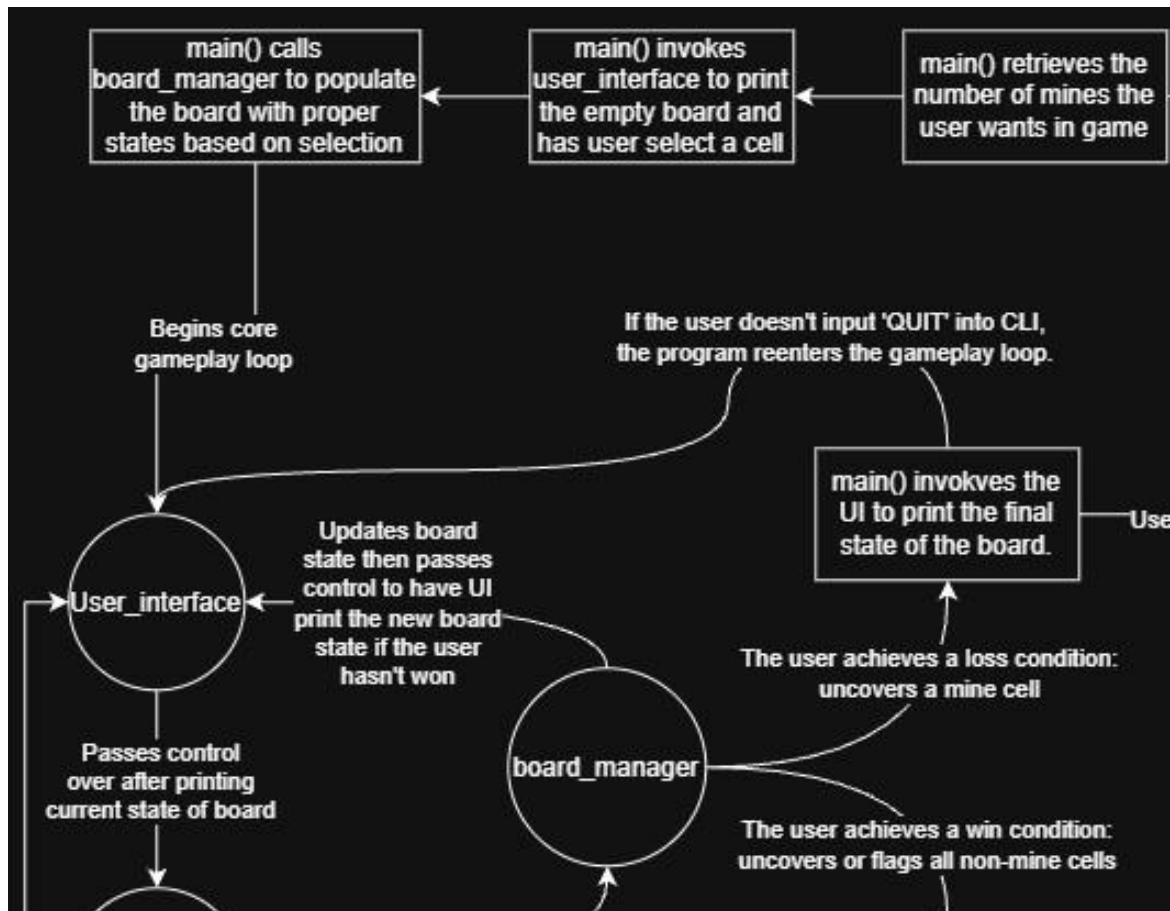
4.1 Overview

Minesweeper is comprised of four or five different modules depending on the game mode: the board manager, user interface, the input handler, AI solver, and the game logic handler. The board manager keeps track of the cell states within the 10 x 10 array and updates them as the game progresses. The user interface displays the grid's current state, the status of the game (such as how many mines and flags are left) along with accepting user inputs. The input handler takes those inputs and sends corresponding outputs to the board manager who will update the board based upon said inputs and the gameplay rules (mine placement, cell uncovering, recursive revealing, etc.). The AI Solver chooses and applies a move on the AI's turn, based on the selected difficulty. This game logic handler creates the loop in which these modules are invoked to create the gameplay loop for the user: starting a game to randomly generate a minefield after clicking into the grid for the first time, uncovering and flagging cells, and ultimately either uncovering a mine and losing or uncovering all non-mine cells and winning.

The game mode determines where each move comes from. In Solo mode, every move comes from the Input Handler. In Multiplayer mode, every move comes from the Input Handler, with the prompt labeled by the current player. In AI mode, the player's moves come from the Input Handler and the AI's moves come from the AI Solver.

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

4.2 Architecturally Significant Design Modules or Packages



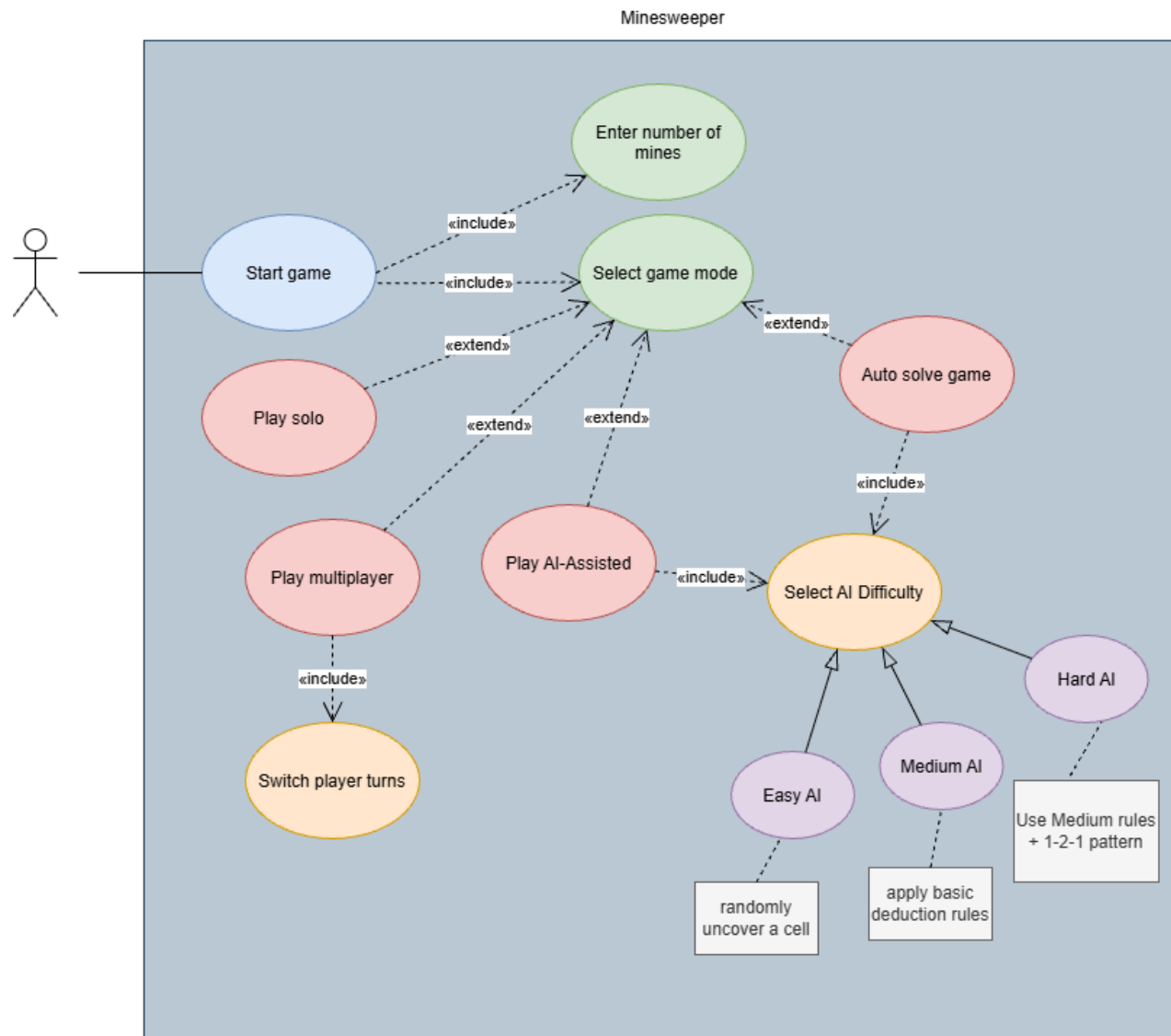
Minesweeper is broken up into the following layered components: the Board Manager or `board.py`, the Input Handler or `input_handler.py`, the User Interface or `user_interface.py`, and the Game Logic Handler or `main.py`.

Before the core gameplay loop of the program is enforced by the Game Logic Handler, the Game Logic Handler gathers how many mines the user would like on the board through Command Line Interface input. The User Interface then prints the newly initialized board, which is all uncovered cells at that moment. The user is then prompted to select what cell they would like to uncover first by the Input Handler. The Input Handler then passes that input to the Board Manager, who then populates the board with mines at random cells that are not the cell that was initially selected by the user. The Board Manager will set the state of the cell selected by the user to be uncovered as well. The User Interface then prints the brand-new set up board for Minesweeper.

The general data flow or core gameplay loop of the program is as follows: the User Interface is invoked to print the current state of the board. The Input Handler then awaits the user to input their action of flagging a cell or uncovering a cell. This input is then passed to the Board Manager which updates the state of the board dependent on the action taken and how that action is affected by the rules of the game. After the board's state update, the user interface prints the new state of the board, and the Game Logic Handler invokes the Board Manager to check if an end state for the game has occurred (if a mine has been

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

uncovered or if all non-mine cells have been uncovered). In which case, it will either display the final state of the board if the user achieved the former end state or it will ask for user input if they would like to play again if the user achieved the latter.



The player begins by starting a game, which always includes two setup steps: entering the number of mines (10 to 20) and selecting a game mode. Each game mode is a variation of game mode selection:

- Play solo: One player plays alone on the board.
- Play multiplayer: Two players alternate turns on the same board, which includes the use case of switching player turns. This is the project's custom addition.
- Play AI-assisted: The player and the AI solver alternate turns on the same board. This mode includes selecting an AI difficulty.
- Auto solve game: The AI solves the board without player turns. It also includes selecting an AI difficulty.
-

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

AI difficulty is Easy, Medium, or Hard. Easy uncovers a random cell, Medium applies the basic deduction rules (flagging and uncovering from revealed numbers), and Hard applies the Medium rules plus the 1-2-1 pattern rule.

4.2.1 Board Manager

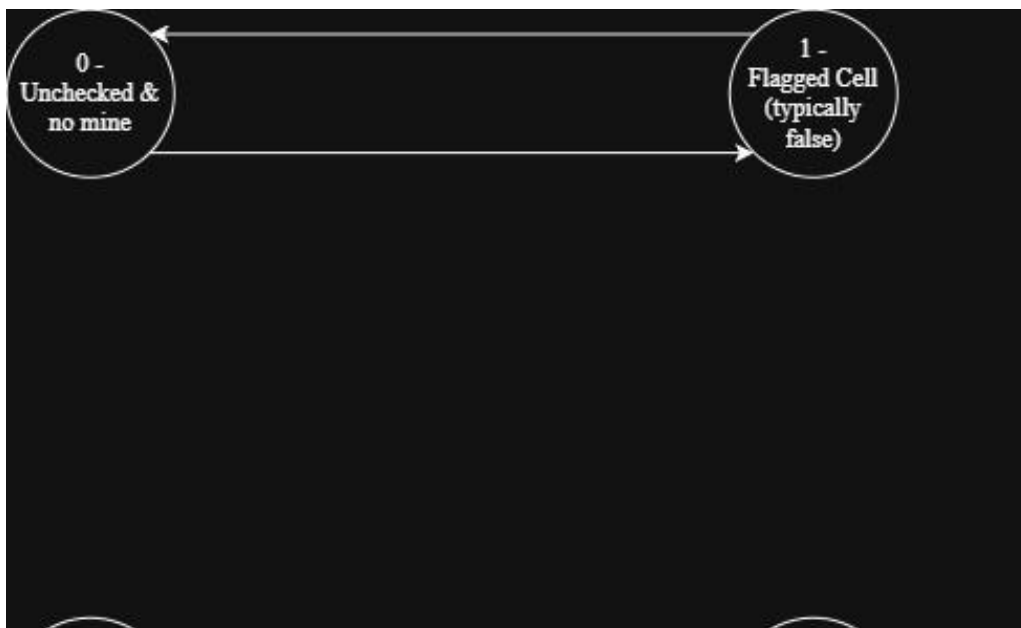
The Board Manager, board.py, is the Board class. It stores the cell grid and enforces the game rules. The class itself has six attributes: rows, cols, numMines, board, gameState, and remainingFlags. While row, cols, and remainingFlags are just integers representing the count of their namesake, board is an integer list representing the current stored state of the Minesweeper grid and gameState is an integer value that tracks whether the player has won, lost, or hasn't achieved either yet.

4.2.1.1 populateBoard(clickedCellRow: int, clickedCellCol: int)

This method is only invoked near the beginning of the program, before the core gameplay loop is invoked by the Game Logic Handler. This method randomly selects a number of cells, equal to the number of mines the user wishes to play with, within the 'board' attribute to have values of 3, representing they are a cell with a mine. Cells that have already been selected to be mines or was the first cell selected by the user to be uncovered are ignored.

4.2.1.2 setState(clickedCellRow: int, clickedCellCol: int, isFlagging: bool = False)

This method is invoked whenever input is handed down to the Board Manager by the Input Handler and a cell's state or value needs to be updated as a result. With the row and column of the selected cell based upon user input, this method changes the cell's state depending on if the user is attempting to flag or uncover a cell. The cell's possible state changes while the user is attempting a flagging action is depicted in the diagram below: each arrow represents the action of the user selecting the state at the beginning of the arrow, ultimately changing the cell's state to be that at the end of the arrowhead.



Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

If the user is uncovering a mine, `getCellState()` is invoked to check if the cell is a mine or an uncovered empty cell. If the cell is a mine, the user loses the game. If the cell isn't a mine, it is changed to be an uncovered cell that displays the number of bombs adjacent to it. If there are no bombs next to the cell, uncovered empty cells around it are recursively uncovered as well.

4.2.1.3 `hasWon()`

This method checks the `gameState` attribute. If it is -1, the player has lost. If it is 0, which the method determines by checking if there are still any uncovered empty cells or falsely flagged cells, the Game Logic Handler will ultimately continue the game. If it is 1, the player has won.

4.2.1.4 `checkAdjCells(row: int, col: int)`

This method returns an integer array that illustrates the state of the cells around a given cell within the Board. It additionally accounts for edge cases where cell states may not exist such as the top or bottom of the grid. This method is primarily invoked to return the number of bombs adjacent to a cell that is uncovered and visible to the player.

4.2.1.5 `numBombNeighbors(row: int, col: int)`

This method returns the number of mines adjacent to a cell, given a cell location. This method is typically invoked by the User Interface when it is printing the board.

4.2.1.6 `getRows()`, `getCols()`, `getCellState()`, `getNumRemainingFlags()`, `getNumMines()`

These methods are used to return the values of the class's attributes.

4.2.2 User Interface

The User Interface class, or `user_interface.py`, is a class designed to display the current state of the board itself. The class itself has four attributes: `myBoard` (an instance of the Board class), `rows`, `cols`, and `board` (a string Array designed to store the symbols of the states of each cell).

4.2.2.1 `print_board(loss = False)`

This method is the only method within the User Interface class and is designed to print the current state of the board for the player.

The method first prints the remaining flags left for the player to utilize, then prints the actual grid itself along with the symbols that denote the state of the cells they are within. The symbols that print and are displayed depend on whether this method is invoked as part of the regular gameplay loop or when the player has lost and the program is displaying the final state of the board.

When `print_board()` is called for the general gameplay loop, the symbols are as follows:

- 'X' – Unchecked Cell
- 'F' – Flagged Cell
- ' ' – Checked Cell with No Mine

When `print_board()` is called when the player causes a losing end state, the symbols are as follows:

- 'X' – Unchecked Cell

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

- 'M' – Mine
- 'F' – Flagged Cell
- '/' – Falsely Flagged Cell
- ' ' – Checked Cell with No Mine

4.2.3 Input Handler

The Input Handler, or `input_handler.py`, is the component meant to detect user input and parse it into commands to pass over to the Board Manager, thus triggering state updates within said board.

4.2.3.1 `get_input()`

The Input Handler contains this singular method. This singular method retrieves two inputs from the user: the cell location they'd like to interact with and whether they'd like to left click (uncover a cell) or right click (flag a cell).

For the first input, the cell location, an input is valid if: the length of the input is greater than 2, if the first index of the input string is in the string 'ABCDEFGHJIJ', if all other elements within the string past the first index is a numerical digit, and if those numerical digits make a number that is between 0 and 9.

For the second input, an input is valid if it is either 'L' or 'R'.

These two inputs are then passed to the Board Manager via way of the Game Logic Handler, though the second input for left and right click is sent as a Boolean value where left is False and right is True.

4.2.4 Game Logic Handler

The Game Logic Handler is primarily a singular function, `main()`, meant to ensure the proper data flow of Minesweeper is ensured. Through the Game Logic Handler, all other components are invoked in the proper order.

Minesweeper's processes are kept active with a 'while True' condition. Inside this condition, the Game Logic Handler first confirms the number of mines the user wants in the game and then initializes the board they will be playing on with randomly generated mine positions along with the user interface they will interact with. It will then keep the User Interface displaying the most up-to-date state of the board as the Input Handler takes the input of the player's actions and passes it to the Board Manager for state updates until the player either wins or loses based upon the actions already previously mentioned earlier in the document.

4.2.5 AI Solver

The AI Solver module, `ai_solver.py`, allows the computer to make moves during AI-assisted game mode. It contains one function, `ai_solver(board, difficulty)`, which receives the current Board and the selected difficulty ("easy", "medium", or "hard") and applies one action to the board.

4.2.5.1 `ai_solver(board, difficulty)`

- Receives the current board and AI difficulty.
- Determines which AI strategy to use.
- Updates the board by uncovering or flagging cells.

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

- Prints the AI's actions so the player can see its moves.

4.2.5.2 Easy AI

- Scans the board for covered cells.
- Excludes flagged and already uncovered cells.
- Randomly selects one available cell.
- Uncovers the selected cell.
- Displays the selected cell's coordinates.
- Does not use logical rules to avoid mines. The AI can lose the game

4.2.5.3 Medium AI

- Examines uncovered cells and their adjacent mine counts.
- Applies two logical rules to determine possible moves.
- Flagging rule: Attempts to identify hidden mines using the numbers displayed on uncovered cells.
- Uncovering rule: Uncovers hidden neighboring cells when the required mines have already been flagged.
- Randomly uncovers an available cell if neither rule applies.
- Displays the AI's actions in the terminal.

4.2.5.4 Hard AI

- Applies the Medium rules first.
- 1-2-1 rule: If three adjacent checked cells in a line show 1, 2, 1, and the three cells beside them on one side are hidden, the two outer hidden cells (next to the 1s) are mines and are flagged. The middle hidden cell (next to the 2) is safe and is uncovered. The pattern is checked in both horizontal and vertical orientations.
- If no rule applies, the AI uncovers a random hidden cell.

Minesweeper Software Architecture Document	Version: 3.0
01 - Software Architecture	Date: 11/OCT/26
SAD	

5. Interface Description

The interface is built utilizing a command-line interface where players enter a cell in format of '[letter][row number]' and then enter 'L' or 'R' depending on whether they are uncovering or flagging the cell whose coordinates they inputted respectively as part of the central gameplay loop.

These input requests are accompanied by prints in which the remaining flags are printed above a grid of cells that display the state of cells currently on the board. Below that grid is the current status of the game: playing, lost, or won.

Before the gameplay loop begins, the player is prompted for the number of mines (10 to 20), the game mode (Solo, Multiplayer, AI, or Auto), and, in AI mode and Auto mode, the AI difficulty (Easy, Medium, or Hard). In multiplayer each cell prompt is labeled with the current player, for example "Player 1 - Enter a cell." An invalid input prints an error message and prompts the same player again. In AI mode, after the user's move the AI automatically makes their move. In Auto, the player chooses the AI difficulty, and the AI plays the entire game without further input.

6. Quality

The simple-to-use user interface as handled by the User Interface component along with the Input Handler ensures the user has the most optimal and smooth experience playing Minesweeper. Simple yet effective input verification makes up most of the Input Handler, ensuring the player can't accidentally use invalid inputs to hardlock their game into an unplayable state.

Python was utilized to develop this game, making it easily portable and compilable for running on any OS that supports Python programming.